Persönliche Videospielverwaltung

Projektbeschreibung

Mein Projekt ist eine Desktop-Anwendung, mit der man seine persönliche Videospielsammlung verwalten kann. Hier können Nutzer ihre Spiele eintragen, deren Status bearbeiten und einsehen (in Besitz, auf der Wunschliste, verkauft, etc.). Ebenso gibt es die Möglichkeit, Spiele zu bewerten und Sammlungen mit mehreren Inhabern anzulegen.

- **Hauptaufgabe**: Zentrale und kompakte Übersicht über die eigenen Spiele und deren Status.
- **Zielgruppe**: Privatpersonen mit großer und kleiner Spielesammlung.
- **Motivation**: Strukturierte Übersicht über die eigene Sammlung, Spiele auf der Wishlist und verkaufte/besessene Spiele mit personalisierten Statistiken, sowie keine Gefahr des doppelten Kaufens bei großen Sammlungen.

Geplante Funktionalitäten

- **Must Have**:
  - Spiele hinzufügen, bearbeiten und entfernen
  - Statuswechsel (Auf der Wunschliste → gekauft/in Besitz → durchgespielt → besessen/verkauft)
  - Sammlung mit mehreren Personen anlegen (z.B.: Familiensammlung)
  - Sammlung anzeigen und nach Kategorien filtern (Genre, Platform, Releasedate, etc.)
- **Should Have**:
  - Persönliche Bewertung mit kurzer Notiz/Kritik
  - Suchfunktion (Titel, Platform, Entwickler, etc.)
- **Nice to Have**:
  - Statistiken (Wieviele Spiele, Gesamtwert, wieviele duchgespielt, etc.)

Bereits implementierten Funktionalitäten

Grundstruktur

- Folgende Klassen implementiert:
  - Game, Platform, Genre, Developer, ReleaseDate, Status, Review, Player, Wishlist und Price
- Getter- und Settermethoden für alle Klassen implementiert
- toString(), equals() und hashCode() implementiert
- Id-Attribute mit final versehen
- Zweiter Konstruktor in Game.java für WishlistHandler.java angelegt
- Zweiten Konstruktor in Review.java für ReviewHandler.java angelegt

Funktionen

- Funktionen addGame() und removeGame() für Wishlist.java implementiert
- GameComparator.java mit Sortierung nach Game, Price, Developer und ReleaseDate erstellt
- Interface SearchCollection für spätere Suchfunktion erstellt

Datenbanken

- SQLite Verbindung nach Anleitungsschritten erstellt und erfolgreich getestet
- MyDatabase.db erstellt
- Handler-Klassen vollständig implementiert
    - Game-Handler
    - Player-Handler
    - Wishlist-Handler
    - Review-Handler
- Hilfsmethode convertToObject() in allen Handler-Klassen implementiert, um Rohdaten aus der Datenbank in ein passendes Objekt zu konvertieren
- 5 Dummy-Daten pro Tabelle in DummyData.java erstellt und erfolgreich getestet

Änderungen seit der letzten Abgabe

Checkliste von Blatt 2

- ☑ Überprüfen Sie, ob weiterhin alle Tutor*innen sowie Fatih Özgan Zugang zu Ihrem Repository haben
- ☑ SQLite und JDBC heruntergeladen und Verbindung getestet
- ☑ Tabellen erstellt, CRUD-Operationen implementiert und getestet
- ☑ Datenbanken gegen SQL-Injection-Angriffe geschützt
- ☑ Dummy-Daten eingefügt
- ☑ Hinweise zu Umsetzung gelesen und eingehalten
- ☑ README.md aktualisiert und als PDF auf Moodle hochgeladen
- ☑ Ergebnisse auf main-Branch gepusht und eigenständig auf der GitLab Webseite überprüft

Verbesserungen und Überarbeitungen

- README-Datei verbessert
- Package-Struktur verbessert und ausgebaut
- Game.java und Review.java um zweiten Konstruktor erweitert für WishlistHandler.java und ReviewHandler.java

Neue Klassen

- Developer.java, Genre.java, Platform.java, Player.java, Review.java, Status.java, Wishlist.java
- DatabaseConnection.java für Datenbankverbindung
- GameHandler.java, PlayerHandler.java, WishlistHandler.java und ReviewHandler.java mit CRUD-Funktionen für Tabellen
- DummyData.java mit Testdaten für Tabellen

Neue Funktionen

- getter-, setter-, hash-, equals- und toString-Methoden für alles Klassen implementiert
- Hilfsfunktion convertToObject() in jeder Handler-Klasse erstellt
- **In GameHandler.java**: saveGame(), deleteGame(), updateGame(), seachById(), showFullCollection()
- **In PlayerHandler.java**: savePlayer(), deletePlayer(), updatePlayer(), findPlayerById()
- **In WishlistHandler.java**: addGameToWishlist(), removeGameFromWishlist(), showPlayerWishlist(), updateGamePrice()
- **In ReviewHandler.java** : saveReview(), deleteReview(), updateReview(), updateReview()

Datenbankstruktur

- Folgende Tabellen wurden erstellt:
    1. games mit Primary Key gameId → Speichert alle übergebenen Spielinformationen
    2. players mit Primary Key playerId → speichert das Nutzerprofil
    3. wishlist mit Primary Key playerId und gameId → ordnet jedem Nutzer eine Wishlist zu und speichert sie
    4. reviews mit Primary Key reviewId → Speichert Nutzerreviews

Noch zu implementieren

- Statistics.java → Wieviele Spiele, Gesamtwert der Sammlung anhand des Einkaufpreises, etc
- Login.java → **Hierbei bin ich mir noch nicht sicher wie ich das genau umsetzen werde, wird aber noch implementiert**
- GUI
- Nebenläufigkeit